온디바이스AI 시험 한 장 요약

Linux·Python | Computer Vision | 딥러닝·GPU — 이론 + 예상 문제

1. Linux

- CLI = 명령어로 조작 / GUI = 아이콘·마우스. **터미널** = 입력하는 창, **셸(bash)** = 명령을 해석·실행
- `user@host:~/a/b$` — `~` 홈, `$` 일반 사용자, `#` root
- `cd` 홈으로 · `cd ..` 상위 · `pwd` 현재 경로 · `ls -a` 숨김 파일 포함
- `mkdir -p a/b` 상위 폴더까지 생성 · `touch` 빈 파일 · `cp` 복사 · `mv` 이동·이름 변경
- `rm` 파일 · `rm -r` 폴더째 삭제 · `rmdir` 빈 폴더만 삭제
- `>` 덮어쓰기 · `>>` 이어쓰기 · `echo $USER`
- `chmod` : r=4, w=2, x=1 (소유자/그룹/기타) · `+x` 실행 권한 추가
- `ping` 연결 확인 · `hostname -I` IP 확인 · `ssh user@host` 원격 접속
- `sudo apt update` 패키지 목록 갱신 · `upgrade` 실제 업데이트 · `install` 설치

Q. `chmod 644` 와 `755` 의 의미? → 644는 소유자 rw-, 나머지 r-- / 755는 소유자 rwx, 나머지 r-x
Q. `echo hi > t; echo yo >> t` 실행 후 t의 내용? → hi\nyo

2. Python · NumPy

- `int(float("0.8825")*100)` → 88 · f-string `{x:.2f}` · `str(h)[0:5]`
- 길이 10인 리스트: `[-1]` 은 마지막 원소, `[10]` 은 IndexError · `[0:-1]` 은 마지막 제외 · `[::2]` 짝수 인덱스 · `[::-1]` 역순
- 컴프리헨션: `[[i,j] for i in .. for j in .. if ..]` (앞의 for가 바깥 루프)
- `enumerate(l, start=1)` → (번호, 값) · `zip(a, b)` → 짝 · 함께 쓰기: `for i,(n,s) in enumerate(zip(..))`
- dict: `.items()` 순회, 새 키에 대입하면 추가 · JSON: `ensure_ascii=False` · YAML: `safe_dump(allow_unicode=True)`
- NumPy: `astype` · `grid[:2,1:]` · `sum/min/max/mean/argmax` · `flatten` (3,3)→(9,)
- **axis=0 → 열(세로) 방향 = 과목별, axis=1 → 행 방향 = 학생별**
- `(a==90).sum()` 개수 · `np.where(a==90)[0]` 인덱스 · 정규화 `(x-min)/(max-min)`

Q. `[1,2,3,4,5]` 를 min-max 정규화하면? → [0, .25, .5, .75, 1]
Q. (5,3) 성적표에서 최고 점수의 학생·과목 찾기 → `i = flat.argmax()` , 학생 = `i // 3` , 과목 = `i % 3`
Q. `"zg0505_짱구".split("_")[1]` → "짱구"

3. 이미지 기본 · 색공간

- 이미지 = **H×W×C 배열**, 원점 **좌측 상단**, 기본 타입 **uint8 0~255**. float로 바꾸면 **/255.** (안 하면 clipping 경고)
- 슬라이싱 = crop · `rotate` : CW=0, 180=1, CCW=2 · `flip` : 1 좌우, 0 상하, -1 둘 다
- **OpenCV = BGR, Matplotlib·skimage = RGB** → `cvtColor(img, COLOR_BGR2RGB)`
- **Gray = 0.299R + 0.587G + 0.114B** (사람 눈이 G에 가장 민감)
- **HSV**: 색상·채도·명도가 독립 → 색 추출에 유리. H는 각도라서 **순환** (360 → 0)

	H	S, V
일반	0~360°	0~100%
OpenCV uint8	**0~179** (360/2)	0~255
OpenCV float32	0~360	0~1

- **LAB**: L* 지각 밝기 0~100, a* 초록↔빨강, b* 파랑↔노랑. 대립색 이론 기반. 밝기와 색이 분리 → **조명 오염에 강함**
- `split` → 채널 조절 → `np.clip` → `merge` → 원래 색공간으로 변환

Q. 일반 H 200°는 OpenCV uint8에서? → 100
Q. H를 100° 이동하는 올바른 코드는? → `h = (h+100) % 360` (`h += 100 % 360` 은 순환이 안 됨)
Q. RGB 이미지를 BGR2GRAY로 변환하면 결과가 다른 이유? → R·B 가중치가 뒤바뀜

4. 필터링

- 커널 합성곱으로 노이즈를 제거하거나 특징을 골라냄. 커널 크기는 **홀수**
- **Smoothing (LPF)**: `blur` = Box (단순 평균) / `GaussianBlur(img,(k,k),0)` = 중심 가중치가 커서 더 자연스러움. 커널이 클수록 더 흐림
- **Sharpening**: 전용 함수 없음 → `filter2D(src, ddepth, kernel)` , ddepth=-1이면 원본과 같은 타입. 예: `[[0,-1,0], [-1,5,-1],[0,-1,0]]` (합 = 1 → 밝기 유지)
- borderType: CONSTANT (0으로 채움, 결과가 확연히 다름) / REFLECT / REFLECT_101 / REPLICATE

5. 주파수 · Edge

- **저주파** = 하늘·피부·기본 형태 (없애면 윤곽선 스케치) / **고주파** = 경계·질감·노이즈 (없애면 흐림)
- LPF = Box, Gaussian / HPF = Residual, Unsharp, Sobel, Canny
- **Residual** = 원본 − 블러 (`subtract`) → 고주파 / **Unsharp Mask** = 원본 + 고주파 (`add`) → 경계 강조
- **Sobel**: 1차 미분. **dx=1 → 세로 edge, dy=1 → 가로 edge**. 사용법 `Sobel(src, CV_64F, dx, dy, ksize=3)`
- magnitude = $\sqrt{(dx^2+dy^2)}$ → `convertScaleAbs` 로 일반 이미지화. 장점: 빠름 / 단점: 노이즈에 약하고 edge가 두꺼움
- **Canny**: ① Gaussian (노이즈 제거) → ② Sobel (edge 후보) → ③ **NMS** (gradient 방향 국소 최대만 유지 → 얇게) → ④ **Hysteresis** (임계값 2개, 중간값은 연결된 경우만 유지)
- `Canny(img, 50, 150)` — **uint8 (CV_8U)만 가능**, float를 넣으면 에러

Q. 커널 `[[-1,0,1]]×3행` 은 어떤 edge를 검출? → 세로 edge (양수 = 왼쪽 어둡고 오른쪽 밝음)
Q. Canny에 /255.로 바꾼 float32를 넣으면? → 에러

6. Segmentation · Morphology

- Color Thresholding → **이진 마스크**. 사용법 `inRange(hsv, lower, upper)`
- **빨강**은 H가 0~10과 170~180 → 마스크 2개를 `bitwise_or` . 마스크 전에 약한 블러 (너무 세면 정보 유실)
- 결과 확인: `bitwise_and(img, img, mask=m)`

연산	원리	효과
Erosion	모두 1이어야 1 (AND)	흰 영역 축소, 배경 노이즈 제거
Dilation	하나라도 1이면 1 (OR)	흰 영역 확장, 구멍·끊김 연결
Opening	침식 → 팽창	작은 노이즈 제거
Closing	팽창 → 침식	내부 구멍 메움

- `morphologyEx(m, MORPH_OPEN, k, iterations=n)` , 커널은 `getStructuringElement(MORPH_ELLIPSE, (5,5))` . 일반 순서: Opening → Closing

Q. 배경에 흰 점이 흩어져 있을 때 쓸 연산은? → Opening / 공 내부에 검은 구멍이 있을 때는? → Closing

7. CLAHE

- **HE**: 전역 밝기 재분배 → 일부가 과하게 밝거나 어두움 / **AHE**: 격자별 평활화 → 단색 영역 노이즈 증폭 / **CLAHE**: 히스토그램 상한(clip)을 넘으면 잘라 재분배
- 순서: **LAB → L 채널에만 적용** → `merge` → RGB → HSV (LAB에서 HSV로 직접 변환 불가)
- `createCLAHE(clipLimit=3.0, tileGridSize=(8,8)).apply(l)`

8. Contour · 특징점

- `findContours(mask, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE)` : RETR_EXTERNAL은 바깥 테두리만, CHAIN_APPROX_SIMPLE은 꼭짓점만 저장
- 노이즈는 작으므로 `max(cs, key=cv2.contourArea)` 로 가장 큰 영역 선택
- **최소 외접원** `minEnclosingCircle` → 중심, 반지름
- **무게중심** `moments` : **Cx = M10/M00, Cy = M01/M00** (M00 = 면적). 반지름은 못 구함 → 외접원 반지름과 함께 사용
- 특징점 = 주변과 구별되는 지점(코너). 대표 기법: **Harris, SIFT, ORB**
- **Harris**: Sobel → Ix^2, Iy^2, IxIy에 Gaussian → **R = det(M) − α·trace²(M)** → Threshold → NMS. R이 크면 코너, 음수면 edge, 0 근처면 평탄

9. 카메라 · 추적 · MediaPipe

- `ls /dev/video*` · CSI 활성화 `jetson-io.py` · 오류가 나면 `systemctl restart nvargus-daemon`
- 캡처 루프: `VideoCapture(0)` → `ret, frame = read()` → `waitKey(1)&0xFF==ord('q')` → `release()`
- USB 카메라 = MJPG(`jpegdec`) / YUYV, `v4l2src` . 포맷 확인 `v4l2-ctl --list-formats-ext`
- **CSI = Bayer(RG10) → ISP 필요 → nvarguscamerasrc** → `VideoCapture(pipe, CAP_GSTREAMER)` . `appsink drop=true max-buffers=1` 로 지연 감소
- **Kalman**: **예측 → 보정** 반복. 가려지면 예측만 수행. 상태 [x, y, vx, vy], 측정 [x, y] → `KalmanFilter(4,2)`
- 프로세스 노이즈가 크면 측정에 민감 / 측정 노이즈가 크면 예측을 더 믿음
- MediaPipe 손 랜드마크 21개, 손가락 끝 **4, 8, 12, 16, 20**. 좌표는 0~1이라 w·h를 곱함. 관절 각도 ≥ 160°이면 펴진 손가락. 화면 반전 시 Left↔Right 바뀜

10. 퍼셉트론 · MLP

- **$z = w_1x_1 + w_2x_2 + b$** , y = f(z). 가중치 = 중요도, 편향 = 활성화 기준
- 단층 → **선형 분리만** 가능 (AND, OR, NAND, NOR) / **XOR = 비선형 → MLP, XOR = AND(NAND, OR)**
- 단일 뉴런으로 XOR 학습 → 확률 약 0.5, 정확도 50%. 구조가 틀리면 epoch를 늘려도 학습 불가
- XOR MLP shape: X(4,2) · W1(2,4) · b1(1,4) · W2(4,1) · b2(1,1) → 출력 (4,1)

	w	b	출력 00/10/01/11
AND	+	큰 −	0 0 0 1
OR	+	작은 −	0 1 1 1
NAND	−	큰 +	1 1 1 0
NOR	−	작은 +	1 0 0 0
XOR	단층 불가		0 1 1 0

Q. w=(0.5, 0.5), b=−0.7, step 함수 → 어떤 게이트? → AND (1+1일 때만 z=0.3 ≥ 0)

11. 활성화 · 학습

- **Step**: x≥0이면 1 (미분값 0 → 학습 불가) · **Sigmoid** $1/(1+e^{-x})$, 0~1 → 이진 분류 · **ReLU** max(0,x), CNN 기본
- 비선형 활성화 함수가 없으면 층을 쌓아도 결국 하나의 선형 함수
- 손실: 이진 분류는 Sigmoid + **BCE** / 다중 분류는 **CrossEntropyLoss** (logit을 그대로 입력, Softmax 내장)
- 연쇄법칙 **dL/dw = dL/dy · dy/dz · dz/dw** · 경사하강법 **w ← w − η·dL/dw**
- 순전파 = 예측 / 역전파 = 기울기를 거꾸로 계산해서 가중치 수정
- 학습률이 작으면 느림 / 크면 발산·불안정 · Epoch가 적으면 **과소적합** / 많으면 **과적합**
- **배치 1회**: `to(device)` → `zero_grad` → forward → loss → `backward` → `step`
- `zero_grad` 가 필요한 이유: PyTorch는 gradient를 누적함 · 평가 시 `eval()` + `no_grad` (가중치 수정 없음)

Q. w=2, η=0.1, dL/dw=5 → 새 w는? → 2 − 0.5 = **1.5**
Q. CrossEntropyLoss 앞에 Softmax를 넣지 않는 이유? → 내부에 포함되어 있어서

12. CNN

- 공간 구조를 유지하며 커널 합성곱 → **Feature Map** (커널 1개당 1장). 가중치 = 커널 각 칸의 값, 편향 = 약한 특징 무시·증폭
- Feature Map 값: 큰 + = 패턴이 강함 / 0 = 특징 없음 / 큰 − = 반대 방향 패턴
- ReLU = 음수 제거 · Pooling = 크기 축소, 효율 향상 · Flatten + FC = 분류. 사람이 설계한 필터와 달리 CNN은 커널을 **학습**
- 얕은 층 = 선·경계 / 깊은 층 = 질감·형태 → 채널 증가 (64 → 128 → 256)
- **출력 크기 = ⌊(H + 2P − K)/S⌋ + 1**
- **Conv 파라미터 = $C_{in}·C_{out}·K^2 + C_{out}$ · FC = in·out + out**

Q. 32×32, K=5, P=0, S=1 → 28 / 28×28, K=3, P=1, S=1 → 28 / 7×7, K=3, P=0, S=2 → 3
Q. MNIST SimpleCNN: 1×28×28 → Conv 8×28×28 → Pool 8×14×14 → Conv 16×14×14 → Pool 16×7×7 → **784** → 64 → 10
파라미터: 80 + 1,168 + 50,240 + 650 = **52,138** · conv1 weight shape [8,1,3,3]
Q. 배치 shape은? → [64,1,28,28] (N,C,H,W). 이미지 한 장에는 `unsqueeze(0)`

- MNIST: 학습 60k / 테스트 10k, 1×28×28 · CIFAR-10: 학습 50k / 테스트 10k, 3×32×32
- `ToTensor` : (H,W,C) → (C,H,W), 0~255 → 0~1 float32
- **데이터 증강** (RandomCrop, HorizontalFlip): **학습 데이터에만**, 과적합 완화
- **정규화** (채널별 mean·std): **학습·테스트 모두**, gradient 안정화. 결과 범위 약 −2~2 → 시각화할 때 역정규화
- **BatchNorm** = 각 층 출력을 정규화해서 학습 안정화 · **AdaptiveAvgPool** = 256×4×4 → 256×1×1로 FC 파라미터 감소 · **Dropout** = 무작위로 뉴런을 꺼서 과적합 완화
- 모델 저장: `torch.save(model.state_dict())` = 학습된 가중치를 보존

13. GPU (CuPy · PyTorch)

- CPU = 소수의 고성능 코어 (Orin Nano 6개), 순차 처리 / GPU = 다수의 코어 (1024개), 병렬 처리
- NumPy 배열은 RAM(host), CuPy 배열은 VRAM(device). Jetson = **UMA** (물리 칩 공유, 논리 영역은 분리) → **복사 필요**
- CPU → GPU `cp.asarray` · GPU → CPU `cp.asnumpy` · 섞어서 연산하면 에러 · `.item()` → Python 스칼라
- 시간 측정: `cp.cuda.Stream.null.synchronize()` (GPU는 비동기 실행) + **warm-up** (첫 실행 초기화 비용)
- 작은 배열에서는 오버헤드 때문에 GPU가 오히려 느림
- GPU convolution: 출력 픽셀이 아니라 **커널 원소마다 반복**하면서 배열 전체를 병렬로 누적 → 결과는 동일
- PyTorch: Tensor, **autograd**, `nn` , `optim` , `utils.data` , `torchvision`
- `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")` — **모델과 텐서는 같은 device에**

자료 오타 주의: 계단 함수 수식이 0과 1이 뒤바뀌어 있음 (정답: z≥0이면 1) · Sobel dy 항목의 "dx=1"은 오타 (정답: dy=1) · CLAHE 주석은 2.0인데 코드는 3.0